G1Control: Project Summary

Controlling a Unitree G1 humanoid robot with body movements captured by a webcam
CITS3200 Group 21 · The University of Western Australia · Semester 2, 2026

1. Goal

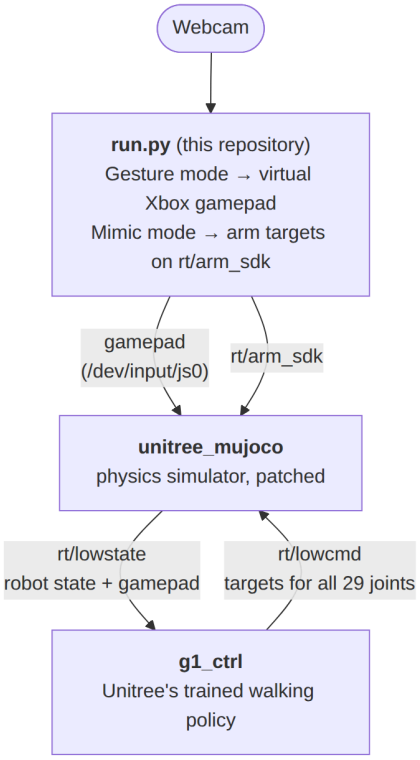
The aim of the project was a control system that lets a person drive a Unitree G1 humanoid robot using only their body, as seen by an ordinary webcam. The robot should respond to the operator's movements: walking in response to hand gestures, and copying the operator's arm movements.

2. What we delivered

Feature	Description	Status
Gesture mode	Eight hand gestures make the robot walk forward or backward, step left or right, and turn left or right. Lowering the hand stops the robot.	Working in simulation
Mimic mode	The robot's arms follow the operator's arms while its legs keep it balanced. The operator switches between modes with the `m` key.	Working in simulation
Real-robot mode	Gesture mode sends walking commands to the physical G1's built-in walking controller over WiFi.	Implemented, not yet tested on a physical robot
Gesture recording tool	Records new samples of each gesture, so recognition can be adapted to a new operator, camera or lighting.	Working
Documentation	`SETUP.md` (installation guide), `USAGE.md` (user guide) and `README.md` (overview, diagrams, glossary and FAQ).	Complete

The system is developed and demonstrated in the Unitree MuJoCo simulator, which uses the same messaging layer (DDS) as the real robot.

3. How it works



In simulation, three programs run together and communicate over DDS:

- `run.py` (our program) reads the webcam. In gesture mode it uses MediaPipe hand landmarks to recognise a gesture and moves the sticks of a virtual Xbox gamepad. In mimic mode it tracks the operator's body pose with MediaPipe, retargets it onto the G1's arm joints with GMR, and publishes arm joint targets.
- `unitree_mujoco` is Unitree's G1 simulator. We patched it so that our arm targets can drive the arms while the legs stay under the walking controller.
- `g1_ctrl` runs Unitree's pretrained walking policy (from `unitree_rl_lab`), which keeps the robot balanced and walks it according to the gamepad input.

On the real robot, the simulator and `g1_ctrl` are not used: `run.py` sends walking commands directly to the robot's built-in walking controller.

Figure 1. System architecture in simulation (also in the README).

4. How we approached it

Weeks 2–4: Understanding the problem and setting up

The team started without prior robotics experience, so the first weeks went into researching the problem, writing the scope of work, a skills and resources audit and a risk register, and writing user stories for the sprints. Everyone set up a common Ubuntu 22.04 development environment, and we documented the procedure so it could be repeated.

Weeks 4–8: Two subteams in parallel

We split into two subteams so both main capabilities could progress at the same time.

- **Gesture recognition.** Built on MediaPipe's hand landmarks. Each gesture is recorded as a set of samples and stored as vectors, and a live hand is matched against them with an adjustable threshold. The subteam ran formal tests of the predefined gestures, then connected recognition to the robot in MuJoCo and tested its responses, including tests using recorded MP4 video.
- **Person identification and pose.** Built on MediaPipe's pose detection. After client feedback in Week 5 asking for real joint data for the robot, the prototype was changed to export MediaPipe's 3D world landmarks (real-world coordinates in metres) for all 33 joints, instead of image-relative coordinates. It was then extended to track a single "leader" among multiple people, and linked to the MuJoCo simulation over CycloneDDS.

Simulation first

To test safely without the physical robot, we worked in simulation from Week 7. We first used plain MuJoCo, then moved to Unitree's `unitree_mujoco` simulator over CycloneDDS, because it accepts the same DDS messages as the real G1. This keeps the path to the real robot short.

Weeks 8–9: Making the robot walk and mimic

- **Walking.** Rather than writing our own balance controller, we used a pretrained walking policy. We first tested Unitree's `unitree_rl_gym` policy, then compared it with `unitree_rl_lab` and adopted the latter (`g1_ctrl`). To connect gestures to it, we built a **gesture-to-gamepad bridge**: recognised gestures are turned into virtual Xbox controller stick movements, which the walking controller already understands.
- **Turning.** Testing showed that turning purely on the spot made the robot wobble rather than turn, so turning was changed to a three-point-turn style that alternates small forward and backward steps.
- **Mimic.** In Week 8 the client asked for the robot to genuinely copy the operator's movements, not just respond to commands. We prototyped inverse-kinematics retargeting with Pinocchio; the final system retargets the operator's arm pose with GMR (General Motion Retargeting). We patched the simulator's DDS bridge so arm targets can drive the arms while `g1_ctrl` balances the legs, and capped arm speed because fast two-arm movements could tip the robot over.

Weeks 9–11: Integration and testing

Once both capabilities worked separately, the subteams were dissolved and the whole team worked on integration and testing. Branches were reviewed and merged through pull requests in an agreed order, a shared interface was added so a single program could switch between gesture and mimic mode with one key, and the combined system was tested end to end in simulation. The code was then cleaned up and commented, and older or outdated code was moved to a separate `Legacy` branch.

Week 11: Documentation and verification

The installation guide was split into separate installation (`SETUP.md`) and usage (`USAGE.md`) guides, and verified by a clean install on a fresh user profile. Following feedback from our mentor, the `README.md` adds diagrams of the architecture, operating modes and start-up sequence, a gesture reference with photos, a glossary and an FAQ.

5. Key decisions

Decision	Reason
Simulation first	Safe, repeatable testing without risk to the physical robot.
Unitree's MuJoCo simulator over DDS	Uses the same messaging layer as the real G1, so the same program can drive both.
Pretrained walking policy (`unitree_rl_lab`)	Reliable balance and walking without training our own controller.
Gestures sent as virtual gamepad input	The walking controller already accepts gamepad commands, so no changes to it were needed.
3D world landmarks for pose	Client feedback asked for real joint data rather than image-relative positions.
GMR for arm retargeting	Converts a human arm pose into G1 joint angles despite the different body proportions.
Pinned versions of third-party code	Reproducible installs; the simulator patch only works with one specific simulator version.

6. Limitations and future work

- **Real robot untested.** Real-robot mode has not yet been run on a physical G1. Testing it, with the safety measures described in `USAGE.md`, is the most important next step.
- **Mimic mode is simulation-only.** Extending it to the physical robot would be a natural extension.
- **Recognition depends on conditions.** Accuracy varies with the operator, camera and lighting; gestures can be re-recorded with the included tool.
- **Platform requirements.** The system needs Ubuntu 22.04 with an Xorg desktop session, and the real-robot network interface is currently set by editing one line of code.
- **Possible improvements.** A recorded-video input for repeatable end-to-end testing, and helper scripts for one-off setup and verification steps.

7. Where to find more

Source code and documentation: **github.com/A5troGaze/CITS3200_Project** (branch `main`).
Start with `README.md` for an overview, then `SETUP.md` to install and `USAGE.md` to run. A short video demonstration accompanies this summary.